“Hermes System”

A Project Report

Submitted in partial fulfilment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Aarish Ansari

Gr No-89

Under the esteemed guidance of

Mr. Nadeem Shaikh

Lecturer

DEPARTMENT OF INFORMATION TECHNOLOGY

K.H.M.W COLLEGE OF COMMERCE

(AFFILIATED to UNIVERSITY of MUMBAI)

MUMBAI,400102

MAHARASHTRA

2025-2026

K.H.M.W. COLLEGE OF COMMERCE

OSHIWARA, JOGESHWARI(W), MUMBAI-400102

DEPARTMENT OF INFORMATIOTECHNOLOGY

APPROVAL OF THE PROJECT PROPOSAL

PRN No.: _____ G.R. No.: _____ Roll No.: _____

Name of the Student: _____

Title of the Project: _____

_____

Name of the Guide: _____

Is this your first submission: Yes ☐ No ☐

_____
(Signature of the Student)

_____
(Signature of the Guide)

_____
(Coordinator)

K.H.M.W. COLLEGE OF COMMERCE

OSHIWARA, JOGESHWARI(W), MUMBAI-400102

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the report of project entitled

_____

undertaken at KHMW College of Commerce by

_____ Seat no.: _____ &

_____ Seat no.: _____ in

partial fulfilment of B.Sc. (I.T.) Degree (Semester VI) Examination (2025–26) has not been submitted for any other examination and does not form part of any other course undergone by the candidate.

It is further certified that they have completed all required phases of the project.

Internal Examiner External Examiner

Coordinator

(College Seal)

ROLES AND RESPONSIBILITIES

1. Project Manager

- Managed overall development of the Hermes system
- Defined project scope, goals, and deliverables
- Created project schedule and ensured timely completion
- Coordinated between all development activities

2. System Analyst

- Analyzed system requirements and user needs
- Prepared requirement specifications and system workflow
- Designed the overall structure of the Hermes system
- Ensured the system meets functional requirements

3. Frontend Developer

- Designed and developed the user interface of Hermes
- Created responsive web pages for users/admin
- Implemented forms, dashboards, and navigation
- Ensured good user experience and usability

4. Backend Developer

- Developed server-side logic of the Hermes system
- Handled authentication, data processing, and APIs
- Managed communication between frontend and database
- Implemented core functionalities of the system

5. Database Administrator (DBA)

- Designed database schema for the Hermes system
- Managed tables, relationships, and data storage
- Ensured data security and consistency
- Performed backup and maintenance

6. Tester / Quality Assurance

- Tested the Hermes system for bugs and errors
- Prepared and executed test cases
- Verified system performance and functionality
- Ensured system reliability before deployment

7. UI/UX Designer

- Designed layout and visual structure of the system
- Created user-friendly interfaces
- Focused on improving user interaction and experience

8. Documentation Specialist

- Prepared complete project documentation
- Maintained reports, diagrams, and system details
- Ensured proper formatting for submission

9. End Users (Admin / Users)

- Admin manages system operations and data
- Users interact with the system features
- Provide feedback for improvements

ABSTRACT

The **Hermes System** is a modern web-based application designed to provide an efficient, reliable, and user-friendly platform for managing system operations and user interactions. The primary objective of this project is to simplify complex processes through automation, improve data management, and enhance overall system performance.

The system is developed using modern web technologies, incorporating a structured frontend, backend, and database to ensure smooth functionality. It allows users to interact with the system through an intuitive interface, while administrators can efficiently manage data, monitor activities, and control system operations. Key features include user authentication, data processing, secure storage, and responsive design.

The Hermes System focuses on improving accuracy, reducing manual effort, and ensuring data consistency. It also emphasizes security measures to protect user information and system integrity. The system is designed to be scalable and flexible, allowing future enhancements and integration with advanced technologies.

Overall, the Hermes System demonstrates the practical implementation of software engineering principles, including requirement analysis, system design, development, and testing. It serves as a reliable solution for efficient system management while providing a strong foundation for future improvements.

ACKNOWLEDGEMENT

It is with immense gratitude and great pleasure that I express my sincere thanks to all those who supported and guided me in the successful completion of my project titled **"Hermes System."**

I am deeply thankful to our respected Principal, **Prof. Abid Jagirdar**, for providing the necessary facilities, encouragement, and a conducive environment to carry out this project successfully.

I would like to express my sincere gratitude to my project coordinator, **Smt. Archana Talekar**, for her constant motivation, valuable suggestions, and continuous support throughout the development of this project.

I am especially grateful to my project guide, **Mr. Nadeem Shaikh**, for his guidance, support, and for helping me resolve my queries, improve documentation, and enhance the overall quality of the project.

I also extend my heartfelt thanks to all the faculty members of the Information Technology department, non-teaching staff, and my friends who directly or indirectly contributed to the successful completion of this project.

DECLARATION

I hereby declare that the project entitled **"Hermes System"**, carried out at **K.H.M.W. College of Commerce**, is my original work and has not been submitted, either in part or in full, to any other university or institution for the award of any degree or diploma.

To the best of my knowledge, this project does not form a part of any work previously submitted by any other individual or group.

This project has been completed in partial fulfilment of the requirements for the award of the degree of **Bachelor of Science in Information Technology (B.Sc. IT)** and is submitted as part of the final semester academic curriculum for the academic year **2025–2026**.

Aarish Ansari

TABLE OF CONTENTS

Sr. No.	Chapter / Section	Page No.
1	**CHAPTER 1: INTRODUCTION**	1
	1.1 Background	2
	1.2 Objectives	3
	1.3 Purpose, Scope and Applicability	4
	1.4 Achievements	6
	1.5 Organization of Report	7
2	**CHAPTER 2: SURVEY OF TECHNOLOGIES**	8
	2.1 Introduction	9
	2.2 List of Technologies	10
	2.3 Comparative Study	12
	2.4 Selected Technologies	13
3	**CHAPTER 3: REQUIREMENTS AND ANALYSIS**	14
	3.1 Problem Definition	15
	3.2 Requirement Specification	16
	3.3 Planning and Scheduling	17
	3.4 Software and Hardware Requirements	18
	3.5 Preliminary Product Description	19
	3.6 Conceptual Model	20
4	**CHAPTER 4: SYSTEM DESIGN**	21
	4.1 Introduction	22
	4.2 System Architecture	23
	4.3 Basic Modules	24

Sr. No.	Chapter / Section	Page No.
	4.4 Data Design	25
	4.5 Procedural Design	26
	4.6 User Interface Design	27
	4.7 Security Issues	28
	4.8 Test Cases	29
5	**CHAPTER 5: IMPLEMENTATION AND TESTING**	30
	5.1 Introduction	31
	5.2 Implementation Details	32
	5.3 Sample Code Snippets	33
	5.4 Testing	34
	5.5 Tools Used for Testing	35
	5.6 Error Handling	36
	5.7 Security Implementation	37
	5.8 Summary	38
6	**CHAPTER 6: RESULTS AND DISCUSSION**	39
	6.1 Introduction	40
	6.2 Results	41
	6.3 System Interface Description	42
	6.4 Discussion	43
	6.5 Future Enhancements	44
	6.6 Summary	45
7	**CHAPTER 7: CONCLUSION AND FUTURE SCOPE**	46
	7.1 Conclusion	47

Sr. No.	Chapter / Section	Page No.
	7.2 Future Scope	48
	7.3 Limitations	49
	7.4 Summary	50
8	**REFERENCES**	51
9	**GLOSSARY**	52
10	**APPENDIX**	53

LIST OF TABLES

Sr. No.	Table No.	Table Name	Page No.
1	Table 2.1	Comparative Study of Technologies	12
2	Table 3.1	Use Case Diagram Symbols	20
3	Table 3.2	Activity Diagram Symbols	21
4	Table 3.3	Data Flow Diagram Symbols	22
5	Table 3.4	ER Diagram Symbols	23
6	Table 4.1	Users Table	25
7	Table 4.2	System Data Table	25
8	Table 4.3	Transaction / Records Table	26
9	Table 4.4	Login Module Test Cases	29
10	Table 4.5	Main Module Test Cases	29
11	Table 4.6	Admin Module Test Cases	30
12	Table 5.1	Test Cases Summary	34

LIST OF FIGURES

Sr. No.	Figure No.	Figure Name	Page No.
1	Figure 3.1	GANTT Chart	17
2	Figure 3.2	PERT Chart	18
3	Figure 3.3	Use Case Diagram	20
4	Figure 3.4	DFD Level 0	22
5	Figure 3.5	DFD Level 1	22
6	Figure 3.6	DFD Level 2	23
7	Figure 3.7	Activity Diagram	24
8	Figure 3.8	Activity Diagram 2	24
9	Figure 4.1	Login Page	32
10	Figure 4.2	Dashboard / Home Page	33
11	Figure 4.3	Main System Interface	34
12	Figure 4.4	Data Management Page	35
13	Figure 4.5	Admin Panel	36

CHAPTER 1: INTRODUCTION

1.1 Background

In today's fast-growing corporate environment, managing employees, tasks, and organizational workflows manually has become inefficient and time-consuming. Many companies still rely on spreadsheets or disconnected systems, which leads to data inconsistency, lack of transparency, and poor communication between teams.

With the advancement of modern technologies, organizations require a centralized system that can efficiently manage employee data, tasks, teams, and projects in a secure and scalable way.

To address these challenges, HERMES – Employee Management System has been developed. It is a full-stack application that allows organizations to manage employees, assign tasks, monitor performance, and handle internal requests efficiently.

The system also includes a desktop application built using Electron, providing a smooth and user-friendly experience. Unlike many modern systems, HERMES is deployed without Docker, using a traditional setup approach for simplicity and accessibility.

1.2 Objectives

The main objectives of the HERMES system are:

- To develop a centralized employee management platform
- To manage employee records, tasks, and projects efficiently
- To enable team creation and collaboration
- To provide task tracking and work monitoring features
- To implement secure authentication using JWT
- To develop a request system for employees (leave, issues, etc.)
- To provide analytics and dashboard visualization
- To ensure data security using encryption and hashing

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The primary purpose of the HERMES system is to simplify employee and organizational management by providing a digital platform that automates tasks, improves communication, and enhances productivity.

1.3.2 Scope

The HERMES system supports the following functionalities:

Employees

- Register and log in securely
- View assigned tasks and projects
- Track work duration
- Submit requests (leave, issues, etc.)

Teams

- Create and manage teams
- Assign members to teams
- Track team performance

Projects

- Create and manage projects
- Assign projects to teams
- Track project status (To Do, In Progress, Completed)

Admin / Management

- Monitor all employees and teams
- Manage system activities
- Analyze performance using dashboards

The system is designed as a desktop-based application (Electron) with a backend server and database, deployed without Docker.

1.3.3 Applicability

The HERMES system can be used in:

- Small and medium-sized companies
- Startups for employee and task management
- Organizations needing internal workflow automation
- Academic projects demonstrating full-stack development

1.4 Achievements

During the development of this project, the following achievements were accomplished:

- Built a full-stack application using Node.js and MongoDB
- Developed a desktop application using Electron
- Implemented JWT-based authentication
- Designed RESTful APIs using Express.js
- Created task and project management modules
- Implemented team collaboration features
- Developed request handling system
- Created dashboard with analytics
- Gained practical experience in real-world system design

1.5 Organization of Report

This report is organized into the following chapters:

Chapter 1 – Introduction

Provides the background, objectives, scope, and purpose of the project.

Chapter 2 – Survey of Technologies

Describes the technologies used in the project such as React.js, Node.js, Express.js, MongoDB, and Bootstrap.

Chapter 3 – Requirements and Analysis

Explains the system requirements, problem definition, and system analysis.

Chapter 4 – System Design

Describes system architecture, modules, database design, and workflow.

Chapter 5 – Implementation and Testing

Explains how the system was implemented and tested.

Chapter 6 – Results and Discussion

Presents system outputs, testing results, and user documentation.

Chapter 7 – Conclusion and Future Scope

Summarizes the project and discusses future improvements.

CHAPTER 2: SURVEY OF TECHNOLOGIES

2.1 Available Technologies

To develop the HERMES – Employee Management System, it was important to select modern and efficient technologies that ensure performance, scalability, and security.

The system is designed as a full-stack desktop application, where different technologies are used for:

- Frontend (User Interface)
- Backend (Server-side logic)
- Database (Data storage)
- Desktop Integration

A study of various technologies was conducted before selecting the final stack.

2.2 List of Technologies

1. HTML (HyperText Markup Language)

HTML is used to create the basic structure of web pages in the system such as login forms, dashboards, and task pages.

2. CSS (Cascading Style Sheets)

CSS is used to style the application and improve the user interface with layouts, colors, and responsive design.

3. JavaScript

JavaScript is used to add interactivity and dynamic behavior such as:

- Form validation
- API calls
- DOM manipulation
- Event handling

4. Node.js

Node.js is used as the backend runtime environment. It allows server-side execution of JavaScript and handles asynchronous operations efficiently.

5. Express.js

Express.js is a web framework for Node.js used to:

- Build REST APIs
- Handle routing
- Manage middleware

6. MongoDB

MongoDB is a NoSQL database used to store:

- Employee data
- Tasks and projects
- Teams and requests

It is flexible and scalable for modern applications.

7. Electron.js

Electron is used to convert the web application into a desktop application. It allows the system to run as a native app on Windows.

8. JSON Web Token (JWT)

JWT is used for secure authentication:

- Generates tokens after login
- Protects API routes
- Ensures secure communication

9. Axios

Axios is used for:

- Sending HTTP requests
- Communicating between frontend and backend

10. VS Code

Visual Studio Code is used as the development environment with features like debugging, extensions, and code management.

2.3 Comparative Study

Technology	Features	Advantages	Disadvantages
HTML	Structure	Easy to use	No logic
CSS	Styling	Improves UI	Depends on HTML
JavaScript	Interactivity	Fast, dynamic	Visible to users
Node.js	Backend runtime	High performance	Callback complexity
Express.js	API framework	Simple routing	Minimal structure
MongoDB	Database	Flexible schema	No joins like SQL
Electron	Desktop app	Cross-platform	Heavy size
JWT	Authentication	Secure tokens	Token expiry handling

Table 2.1: Comparative Study

2.4 Selected Technologies

Based on the analysis, the following technologies were selected:

Frontend

- HTML
- CSS
- JavaScript

These technologies provide a simple and interactive user interface.

Backend

- Node.js
- Express.js

Used to handle server logic, API creation, and request processing.

Databas

- MongoDB with Mongoose

Used for storing and managing application data efficiently.

Desktop Application

- Electron.js

Used to convert the system into a desktop application for better usability.

Authentication & Security

- JWT (JSON Web Tokens)
- bcrypt

Used for secure login and data protection.

Chapter 3: Requirements and Analysis

3.1 Problem Definition

In most organizations, task tracking, scheduling, and productivity monitoring are still performed manually or through unstructured tools such as spreadsheets or email communication. This results in several inefficiencies that affect both management and employees.

Lack of Centralization

- Without a unified system, task information is scattered across multiple sources, making it difficult for administrators to monitor employee activities effectively.

Task Overlaps and Time Conflicts

- Employees may unknowingly record overlapping or duplicate tasks, leading to inaccurate performance tracking and confusion in work reporting.

Limited Visibility and Analysis

- Managers lack real-time insights into work progress, making it harder to identify productivity trends or performance bottlenecks.

Manual Reporting and Low Accountability

- Since tasks are not logged automatically, it becomes challenging to generate performance reports or ensure that employees remain accountable for their assigned work hours.

Data Security Concerns

- Traditional reporting systems often fail to secure sensitive organizational data, increasing the risk of unauthorized access or data loss.

This project addresses these issues by developing a **cross-platform desktop application** that allows users to log and track tasks in real-time, automatically detects scheduling conflicts, and provides administrators with analytical charts for performance monitoring.

3.2 Requirement Specification

The system is designed to streamline the process of recording, analyzing, and managing employee tasks while ensuring data accuracy and security.

3.2.1 Functional Requirements

User Functions

- Register and log in securely.
- Create, update, and delete task entries.
- Add task details including start time, end time, date, and notes.
- View personal productivity history and weekly performance charts.

Admin Functions

- Manage user accounts and access levels.
- View and analyze all users' task data.
- Monitor weekly or monthly work hours through graphical reports.
- Detect and resolve overlapping or conflicting task entries.

3.2.2 Non-Functional Requirements

- **Usability**: Simple, attractive, and user-friendly interface.
- **Performance**: Website should load fast and handle multiple users.
- **Security**: Passwords should be securely hashed; access must be role-based to prevent unauthorized data viewing.
- **Scalability**: The system architecture should support future extensions, such as cloud synchronization or team analytics.
- **Reliability**: Data should be backed up and protected against crashes or accidental deletions.
- **Portability:** The Electron-based app should run seamlessly across Windows, macOS, and Linux.

3.3 Planning and Scheduling

3.3.1 GANTT Chart

Gantt Chart: Used to plan project phases like design, development, testing, and deployment with timelines.

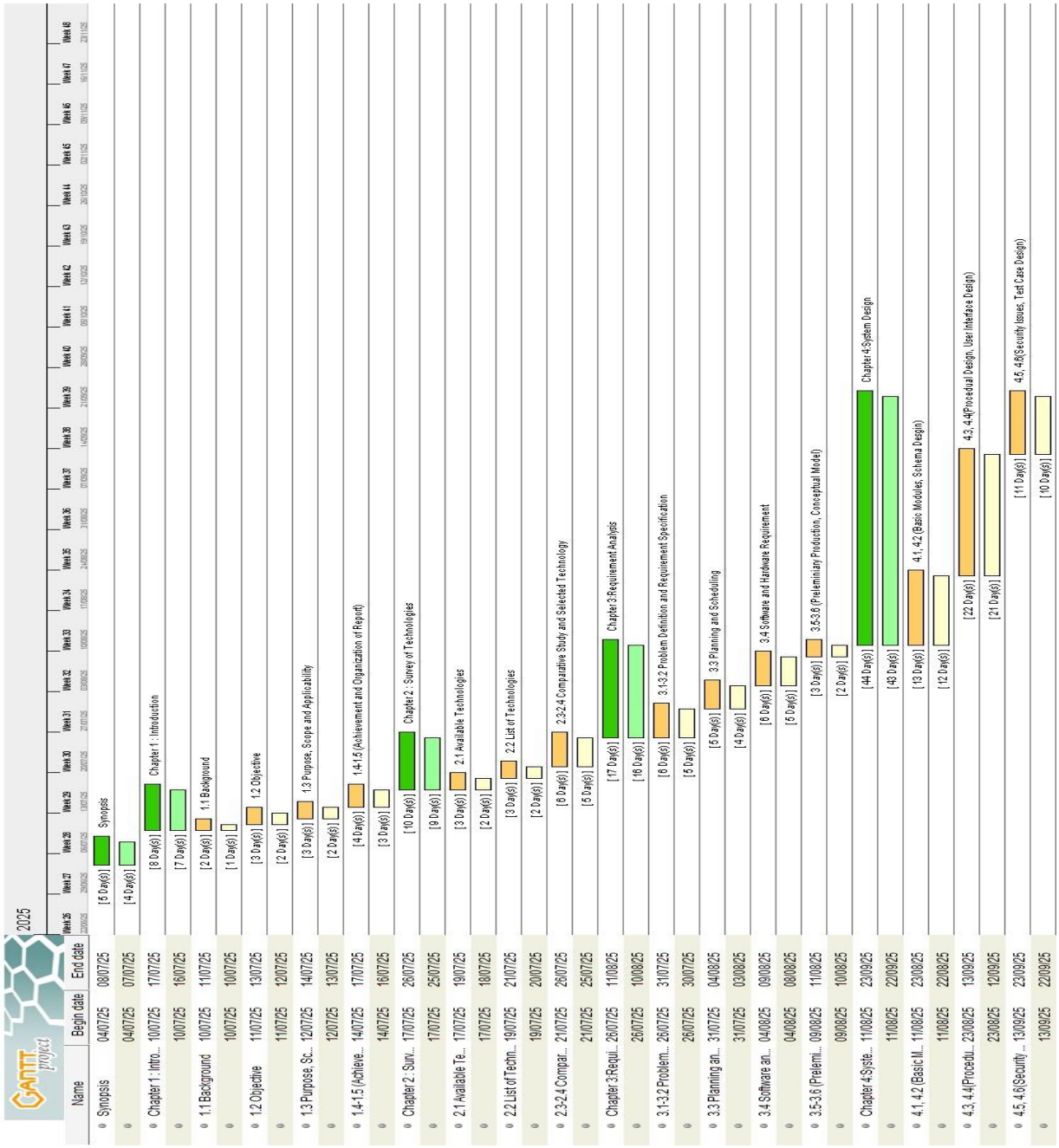


Figure 3. 1 GANTT Chart

3.3.2 PERRT Chart

PERT Chart: Used to identify dependencies between tasks such as product catalog setup, payment gateway integration, and order management.

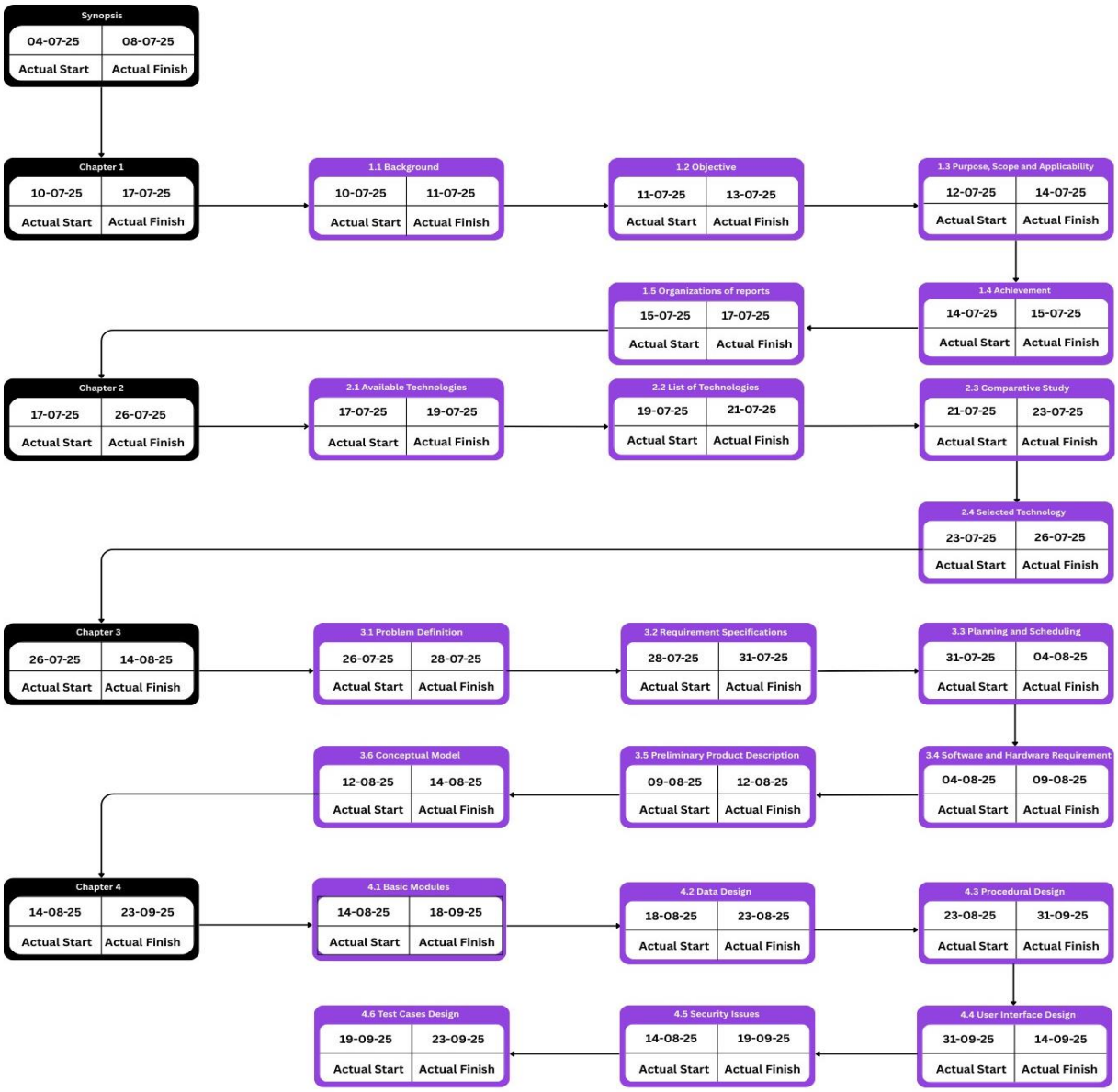


Figure 3. 2 PERT Chart

3.4 Software and Hardware Requirements

3.4.1 Developer Requirements

Hardware

- PC/Laptop with at least 8 GB RAM
- Dual-core processor, and internet.

Software

- OS: Windows / Linux
- IDE: VS Code
- Frontend: HTML, CSS, JavaScript
- Backend: Node.js with Express.js
- Database: MongoDB
- Project Management: Trello
- Deployment: Vercel / Docker

3.4.2 User Requirements

Hardware: Smartphone or PC with internet.

Software: Any modern web browser (Chrome, Firefox, Edge).

3.5 Preliminary Product Description

The Task and Productivity Tracking Application provides an integrated platform for managing daily work logs, detecting time clashes, and visualizing performance metrics.

Users can record tasks with timestamps, notes, and categories, while administrators can view organization-wide analytics through interactive charts. The system enhances accountability, helps identify productivity trends, and reduces the manual effort of maintaining spreadsheets or reports.

This application ensures smooth coordination between team members, provides a secure repository for task data, and supports decision-making with data-driven insights.

3.6 Conceptual Model

3.6.1 Use Case

A Use Case Diagram is a visual representation of the functional requirements of a system, showing how users (actors) interact with the system to achieve specific goals. It's a tool used in software development, systems engineering, and business analysis to identify, clarify, and organize system requirements.

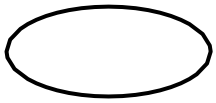
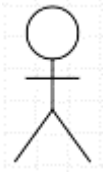
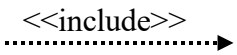
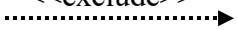
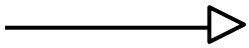
Symbol	Name	Description
	Use Case	These describe the actions performed by actors to achieve a specific goal.
	Actor	These are the users or external systems that interact with the system.
	Include	includes the behavior of another.
	Exclude	It indicates one use case extends the behavior of another.
	Generalization	It indicates that an actor or use case inherits properties from another.

Table 3. 1 Use Case Diagram Symbol

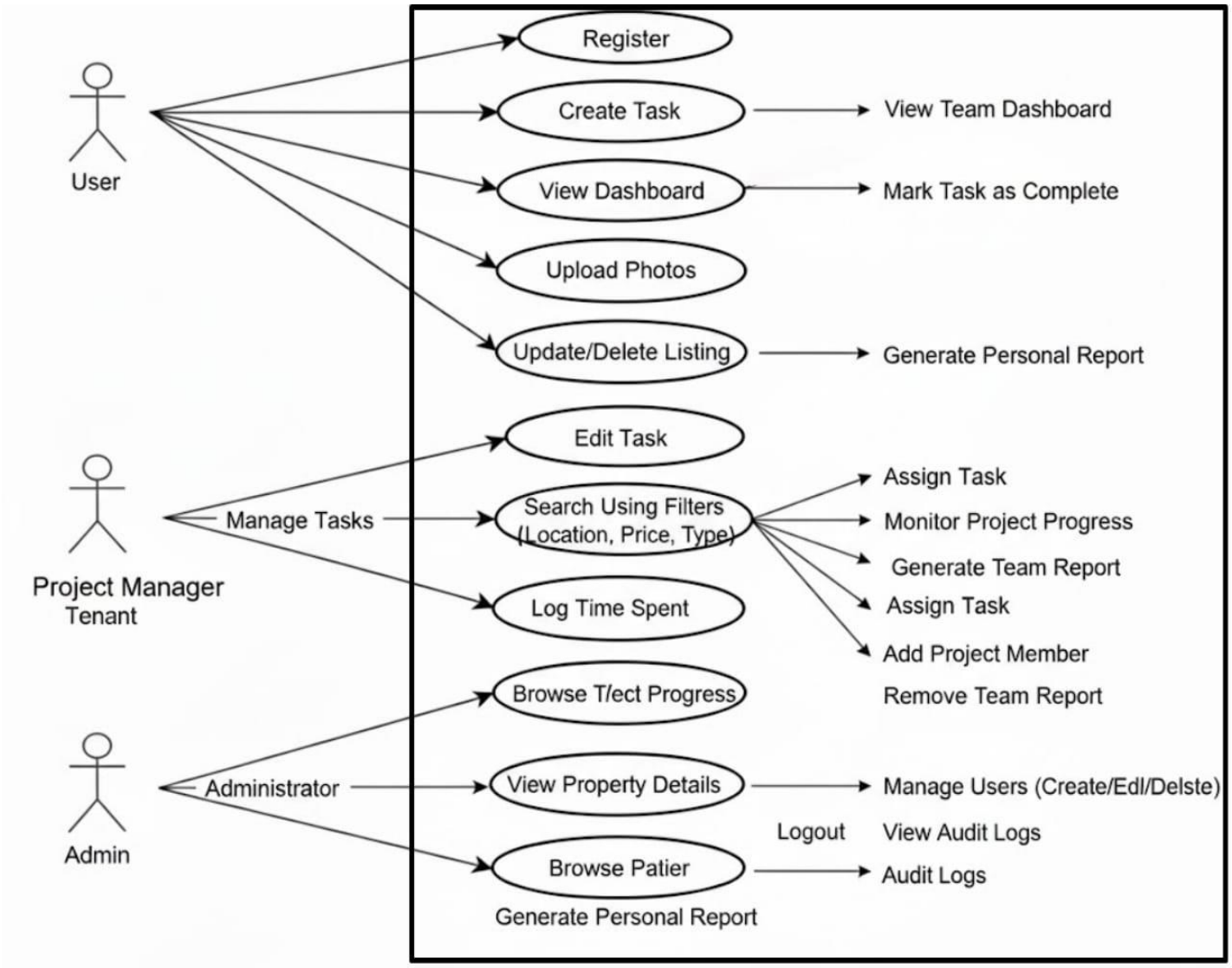
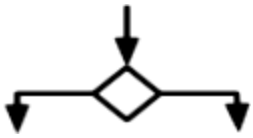


Figure 3.3 Use Case Diagram

3.6.2 Activity Diagram

Activity Diagram is a type of UML (Unified Modelling Language) diagram that shows the dynamic behaviour of a system, highlighting the flow of control and data between activities. It's used to model the workflow, business processes, and operational procedures of a system

Symbol	**Name**	**Description**
	Action	These are the actions or tasks performed within the system.
	Initial State	Activity Diagrams start from this step. Also known as the entry state.
	Final State	This is the last stage of the activity diagram. This is where the activity ends in a software system.
	Decision Node	It represents flow of control is determined allowing the diagram to branch into different paths based on specific conditions.
	Merge Node	It represents the convergence of multiple flows into a single path. It's used to reunite branches that were split by a Decision Node or a Fork Node.

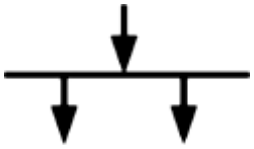
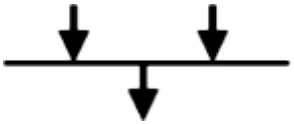
	Fork	It generally includes a single input but may or may not get one output.
	Join	A join is one where tworesults of concurrent activities add and form a single result.
	Control Flow	Connectors between two states or two actions to depict the flow. Shows the sequence of execution.

Table 3. 2 Activity Diagram Symbol

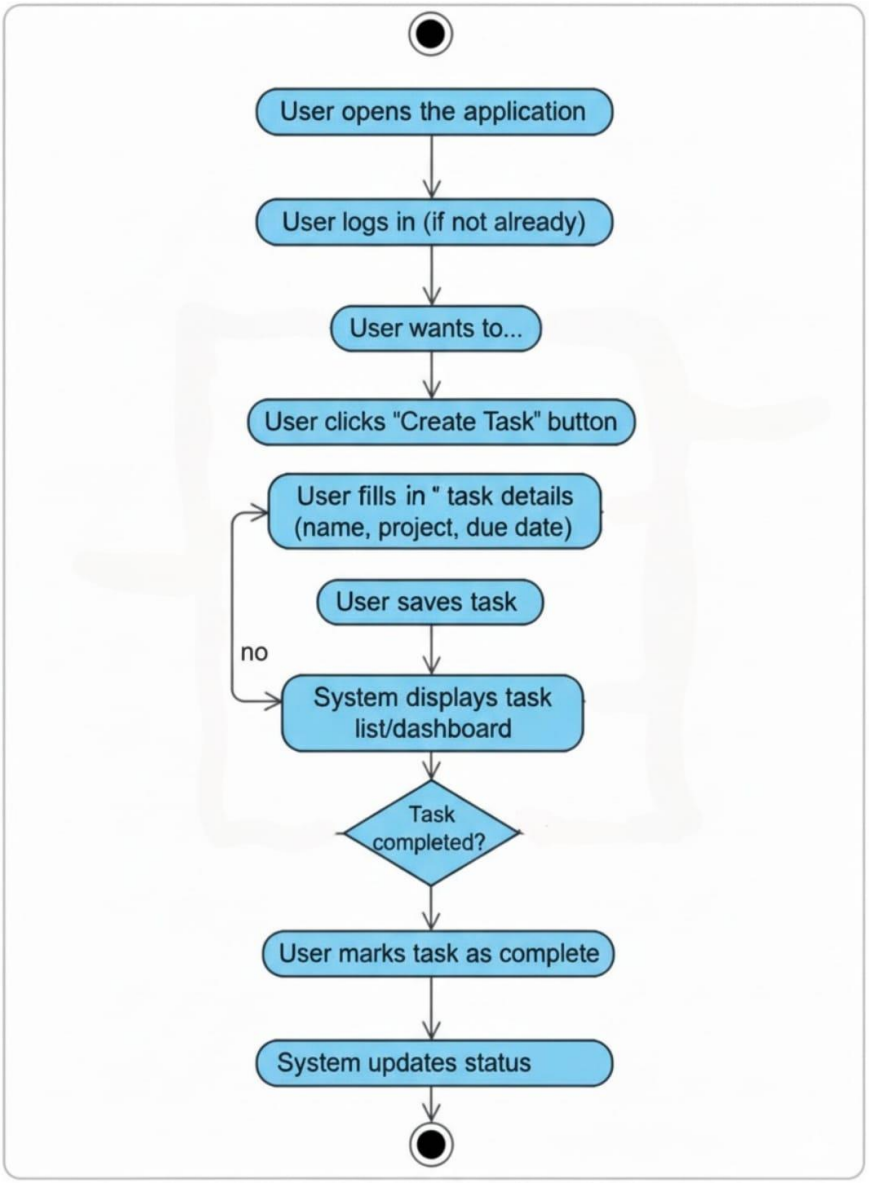


Figure 3. 4 Activity Diagram

3.6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a visual representation of the flow of data through a system, highlighting the inputs, processing, and outputs. DFDs make it easy to depict the business requirements of applications by representing the sequence of process steps by step. It shows how data enters and leaves the system, what changes the information, and where data is stored.

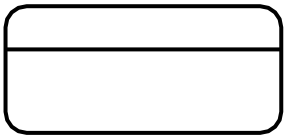
Symbols	Name	Description
	External Entity	These are sources or destinations of data outside the system, such as users, organizations, or other systems.
	Process	Process is a procedure that manipulates the data and its flow by taking incoming data, changing it, and producing an output.
	Data Store	Data stores are files or repositories that hold data for later access such as databases, files, or documents.
	Data Flow	Arrows showing data movement between processes.

Table 3. 3 Data Flow Diagram Symbol

Level 0

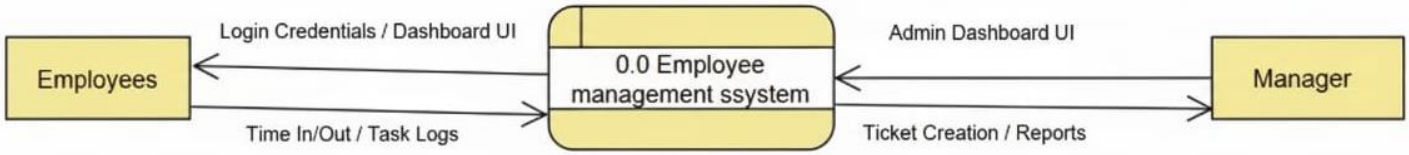


Figure 3. 5 DFD Level 0

Level 1

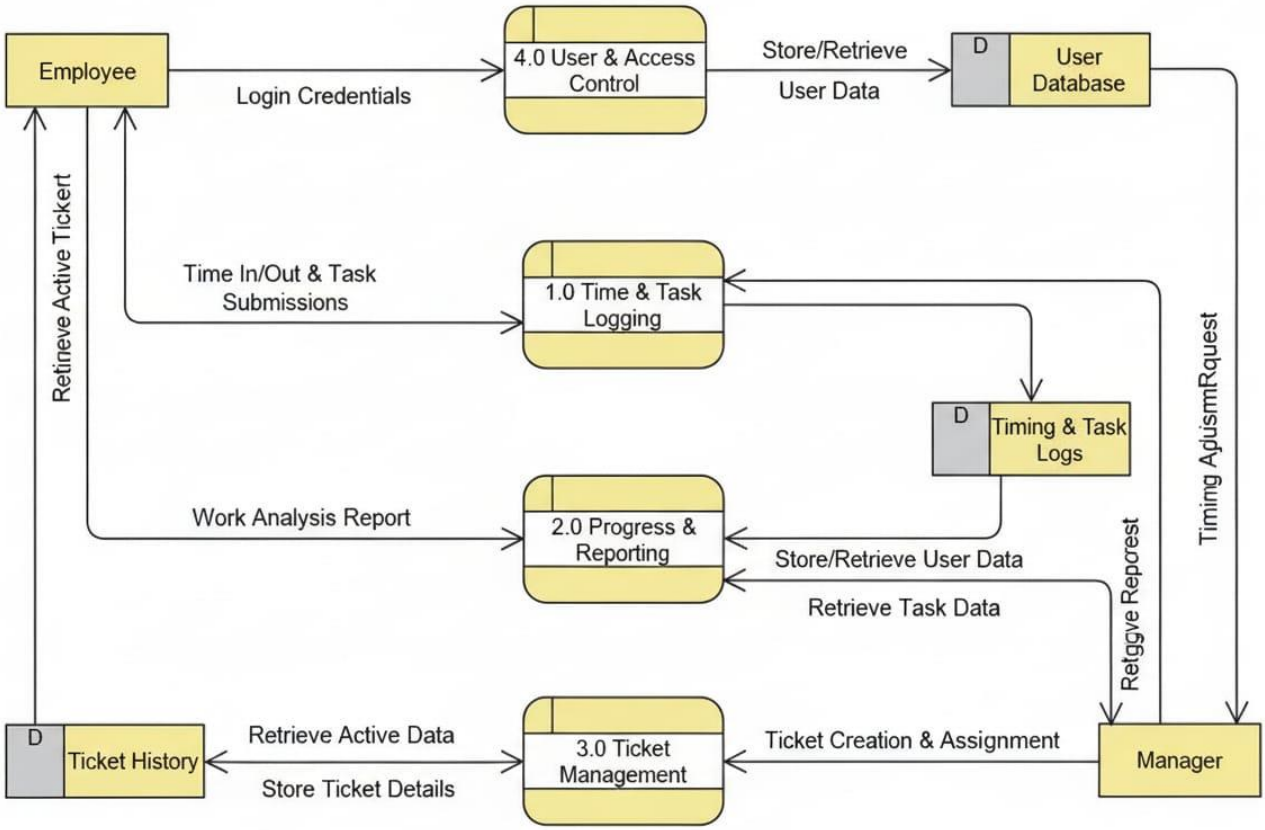


Figure 3. 6 DFD Level 1

Level 2

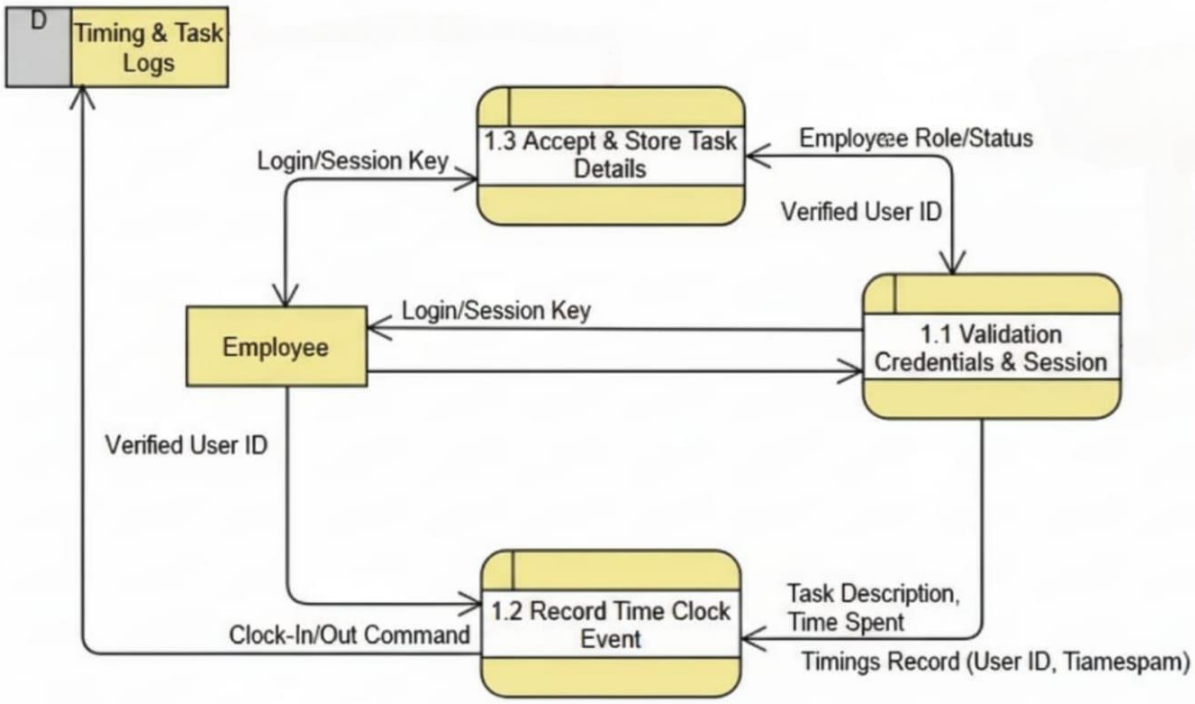


Figure 3. 7 DFD Level 2

3.6.4 ER Diagram

An Entity-Relationship (ER) diagram is a visual representation used to model the structure of a database. ER diagrams use symbols to represent entities, attributes and relationships. ER diagrams are particularly useful for designing and understanding databases and their relationships.

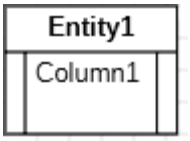
Symbol	Name	Description
	Entity	These are the tables that have data stored about them in the database.
	One-to-one relationship	Single Entity is related to another single Entity.
	One-to-many (1..*) relationship	Single instance of Entity A can be related to multiple instances of Entity B.
	One-to-many (0..*) relationship	Single instance of Entity A can be related to multiple instances of Entity B, but each instance of Entity B is related to only one instance of Entity A.
	Many-to-Many relationship	Multiple instances of an Entity can be related to multiple instances of another Entity.

Table 3. 4 ER Diagram Symbol

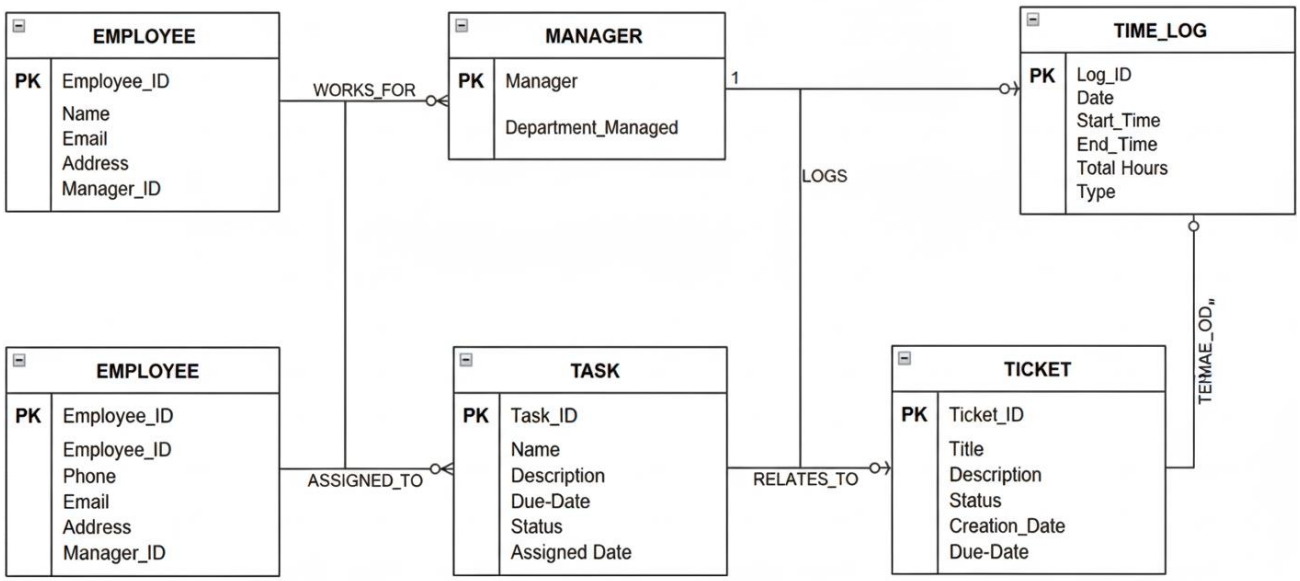


Figure 3. 8 ER Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 Basic Modules

The HERMES – Employee Management System is divided into multiple modules to ensure scalability, maintainability, and efficient data handling. Each module performs a specific function and interacts with others through structured data flow.

Employee Module

- Register and log in securely
- View assigned tasks and projects
- Track work duration
- Submit requests (leave, issues, etc.)

Task Management Module

- Add, view, and delete tasks
- Prevent overlapping tasks
- Track task duration
- Generate weekly work summary

Team Management Module

- Create and delete teams
- Assign employees to teams
- View team members and roles

Project Management Module

- Create, update, and delete projects
- Assign projects to teams
- Track project status:
- To Do
- In Progress
- Completed

Request Module

- Submit employee requests
- Handle leave and issue requests
- Maintain request history

Dashboard Module

- Display task summary (last 7 days)
- Track work time
- Show analytics using charts

Authentication Module

- User signup and login
- JWT-based authentication
- Password hashing using bcrypt

4.2 Data Design

4.2.1 Schema Design

The database is a core component of HERMES, responsible for storing and managing all system data efficiently.

The schema is divided into:

1. Physical Schema

- Collection structure (MongoDB)
- Field types
- Indexing

2. Logical Schema

- Relationships between entities
- Data flow between modules

Database Name: HERMES_DB

Collections (Tables):

- employees
- tasks
- teams
- projects
- requests

4.2.2 Data Integrity & Constraints

Data integrity ensures accuracy and consistency of data.

Types of Integrity:

- **Entity Integrity** → Unique IDs (_id in MongoDB)
- **Referential Integrity** → Linking tasks, teams, projects
- **Domain Integrity** → Valid values (status fields)

Collection Structures (Summary)

Employees Collection

Field	Type	Description
_id	ObjectId	Unique Employee ID
name	String	Employee name
email	String	Unique email
password	String	Hashed password
role	String	Employee role

Table 4.1: Employees Collection

Tasks Collection

Field	Type	Description
_id	ObjectId	Task ID
employee_id	ObjectId	Assigned employee
title	String	Task title
duration	Number	Work time
date	Date	Task date

Table 4.2: Tasks Collection

Teams Collection

Field	Type	Description
_id	ObjectId	Team ID
team_name	String	Team name
members	Array	Employee list

Table 4.3: Teams Collection

Projects Collection

Field	Type	Description
_id	ObjectId	Project ID
name	String	Project name
team_id	ObjectId	Assigned team
status	String	Project status

Table 4.4: Projects Collection

Requests Collection

Field	Type	Description
_id	ObjectId	Request ID
employee_id	ObjectId	Employee
type	String	Leave/Issue
status	String	Pending/Approved

Table 4.5: Requests Collection

4.3 Procedural Design

4.3.1 Logic Flow

The system follows this workflow:

Start

↓

User Login

↓

JWT Authentication Verified

↓

Access Dashboard

↓

Perform Operations

↓

Store Data in MongoDB

↓

Display Results on UI

↓

End

Figure 4.1: Logic flow diagram

4.3.2 Data Structures Used

- **Arrays** → Store lists (tasks, teams)
- **Objects (JSON)** → Store structured data
- **Queues** → Request handling (optional workflow)
- **Counters** → Track analytics

4.3.3 Algorithm (System Workflow)

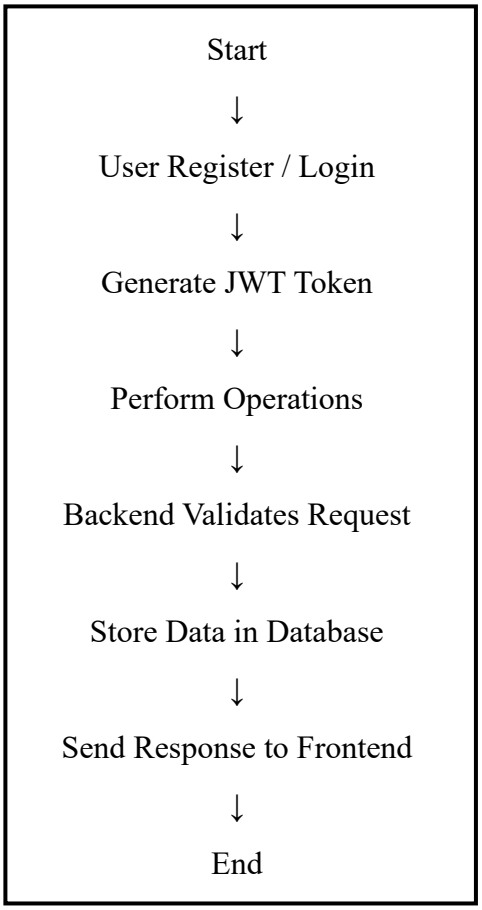


Figure 4.2 System Workflow diagram

4.4 User Interface Design (UI)

The UI is designed to be simple, clean, and user-friendly.

Login page

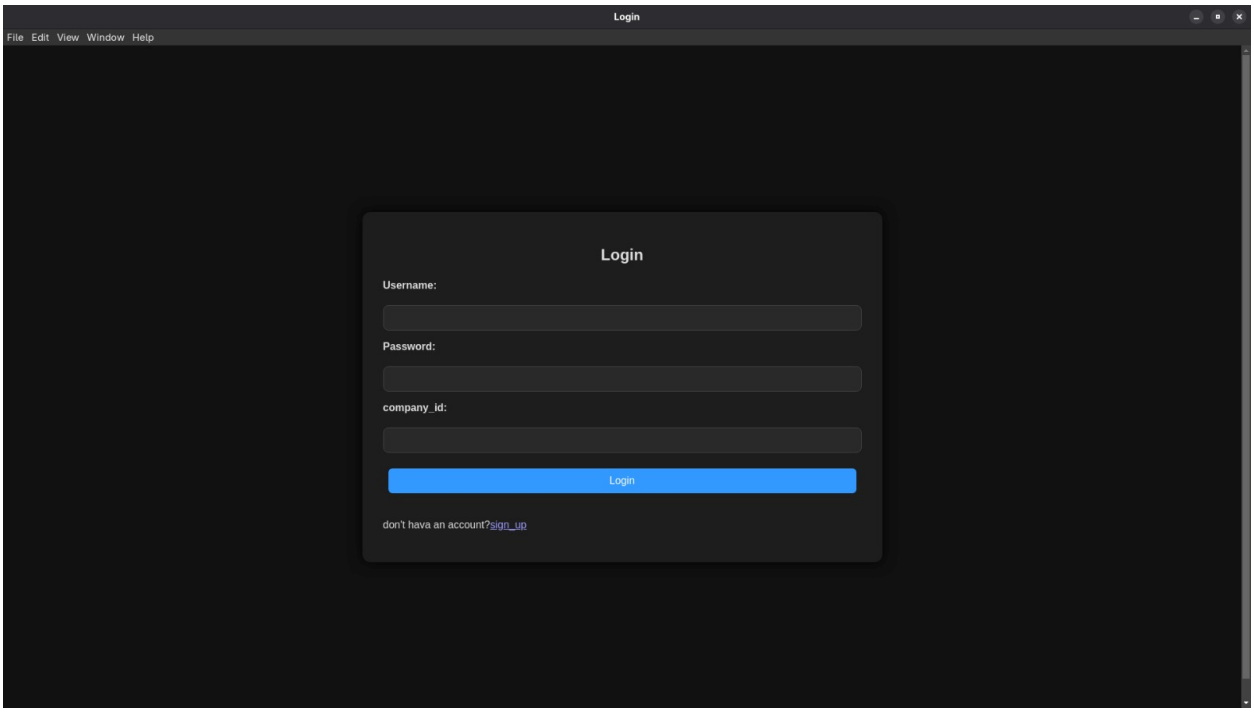


Figure 4.3: Login page

Sign up page

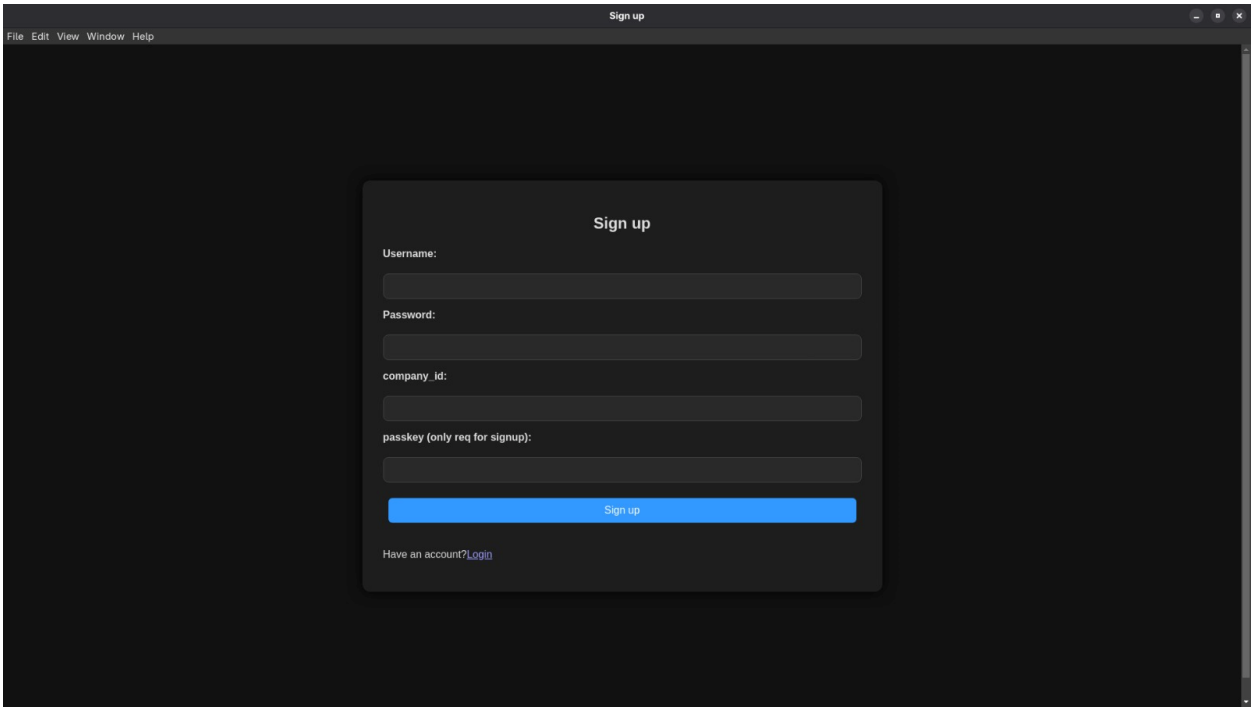


Figure 4.4: Sign up page

Home page

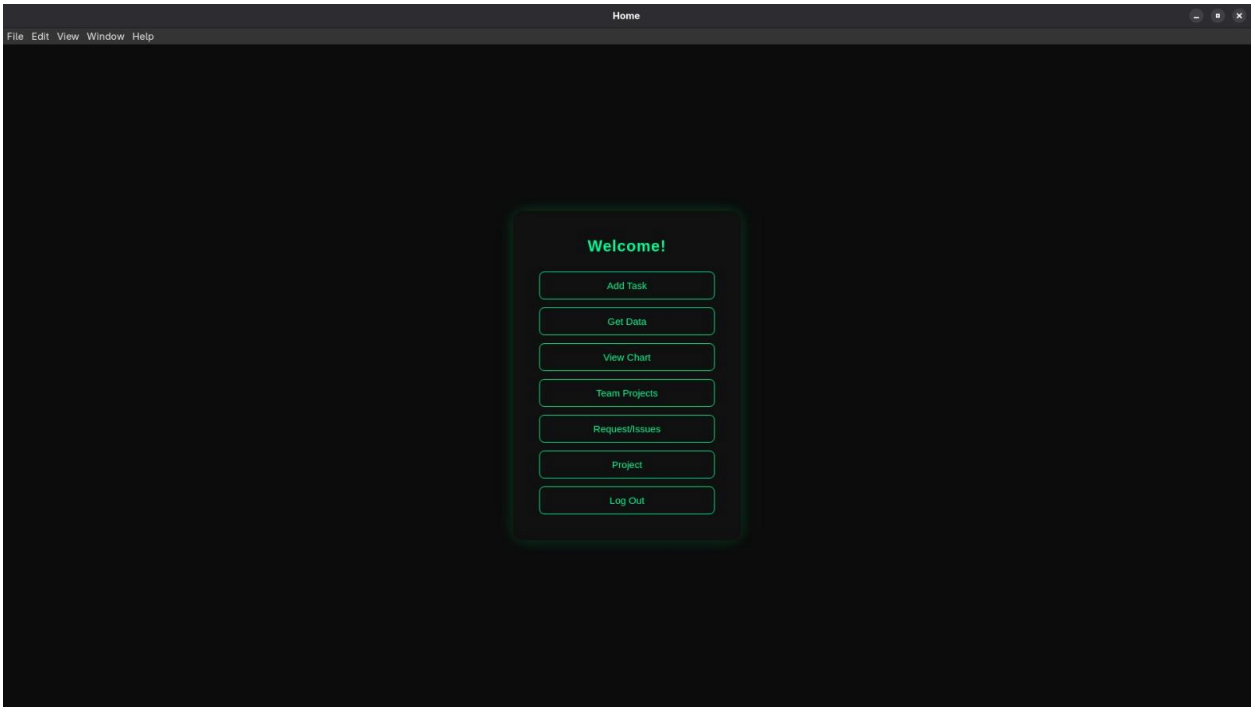


Figure 4.5: Home page

Add Task Page

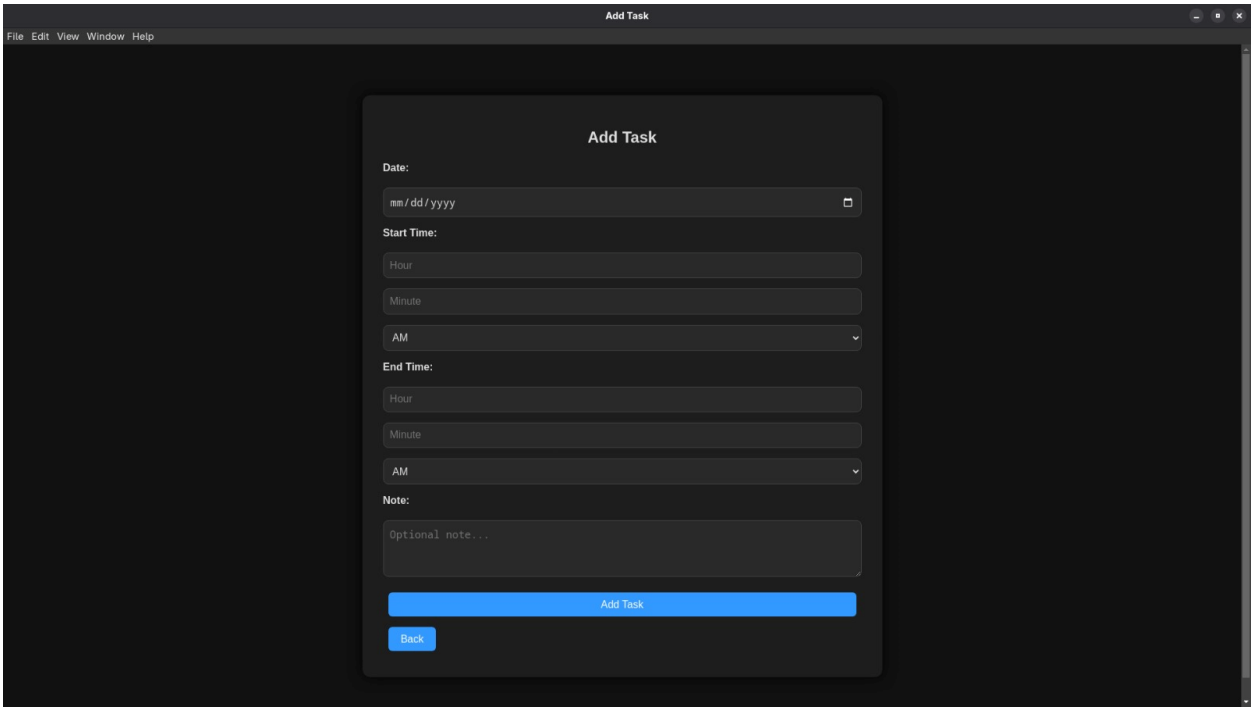


Figure 4.6: Add Task Page

Work Summary Page

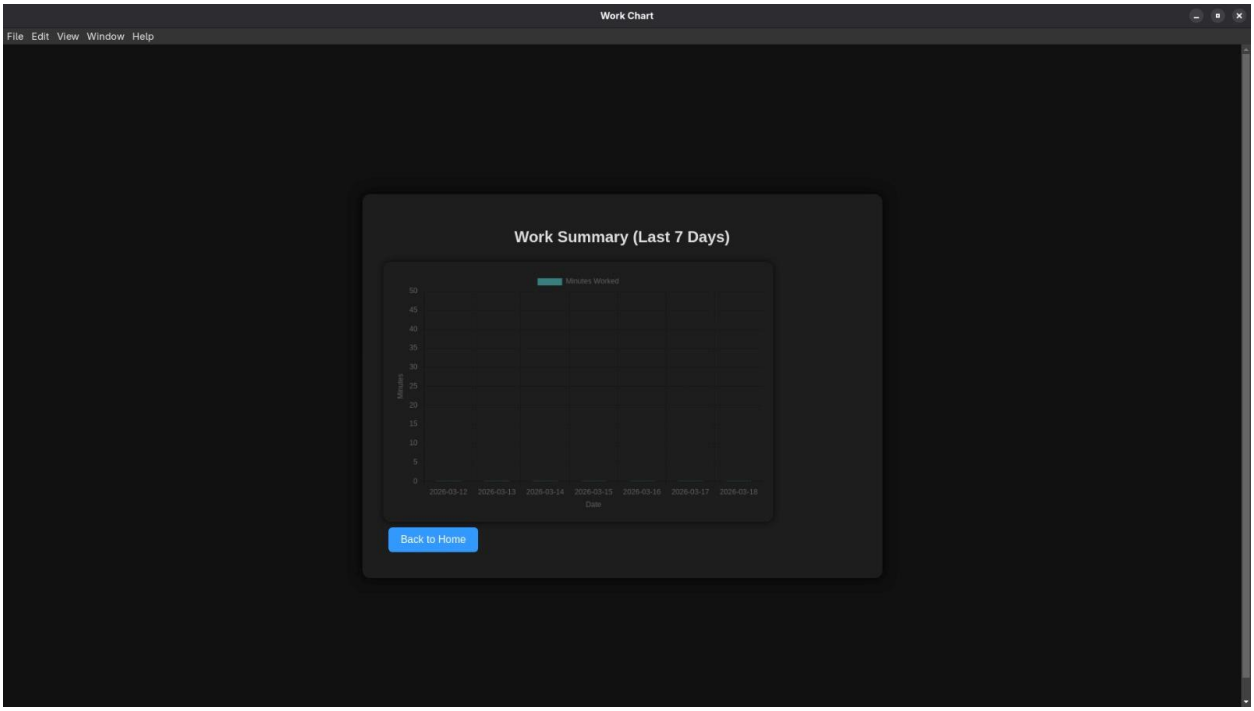


Figure 4.7: Work Summary Page

Teams & Projects Page

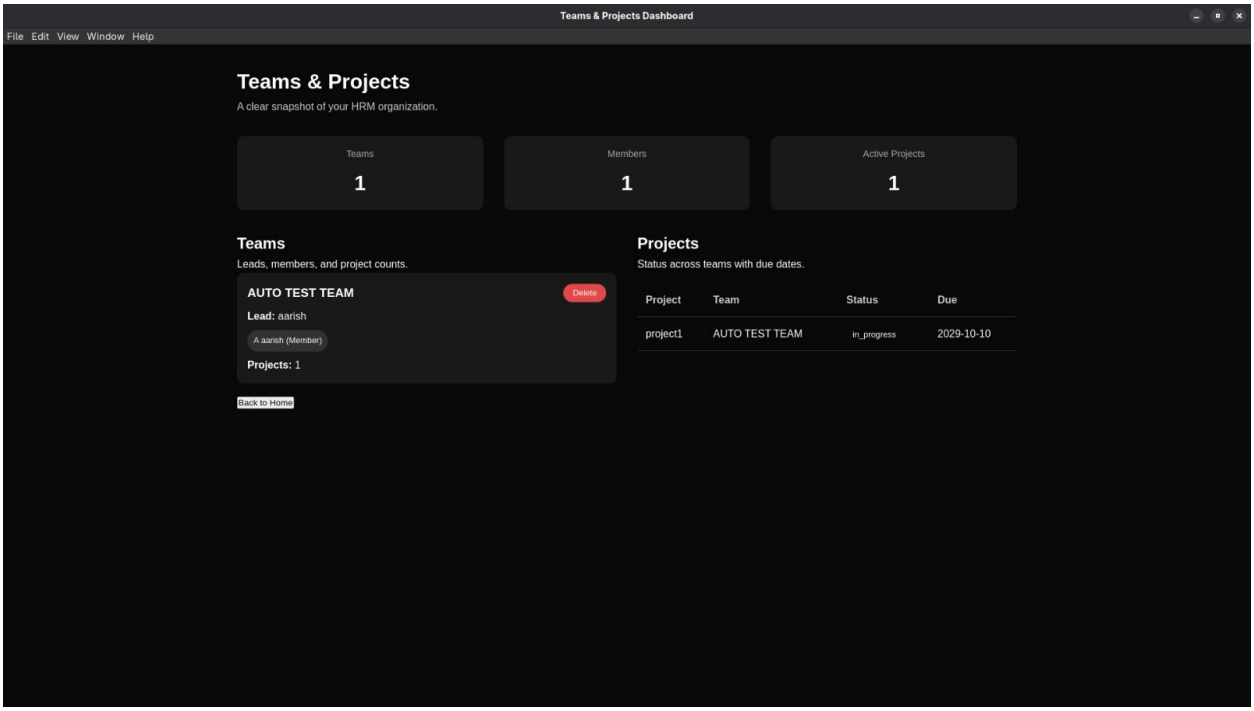


Figure 4.8: Teams & Projects Page

Project Borad Page

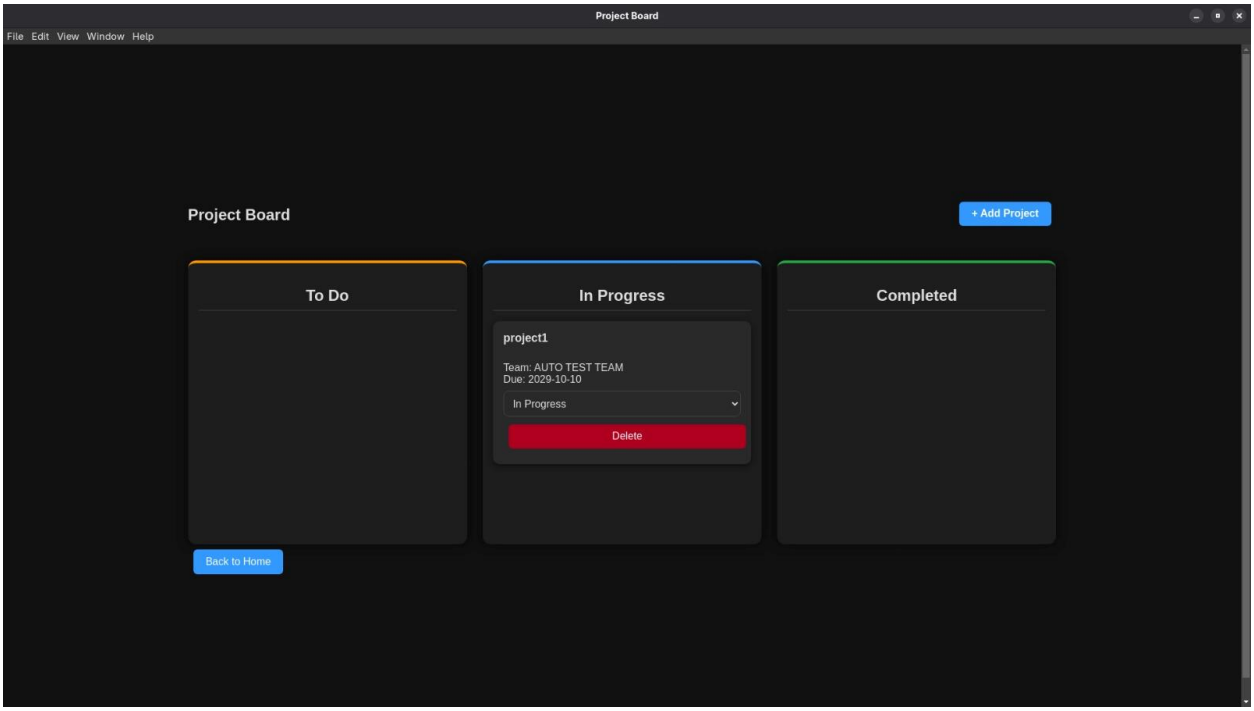


Figure 4.9: Project Borad Page

Send Request Page

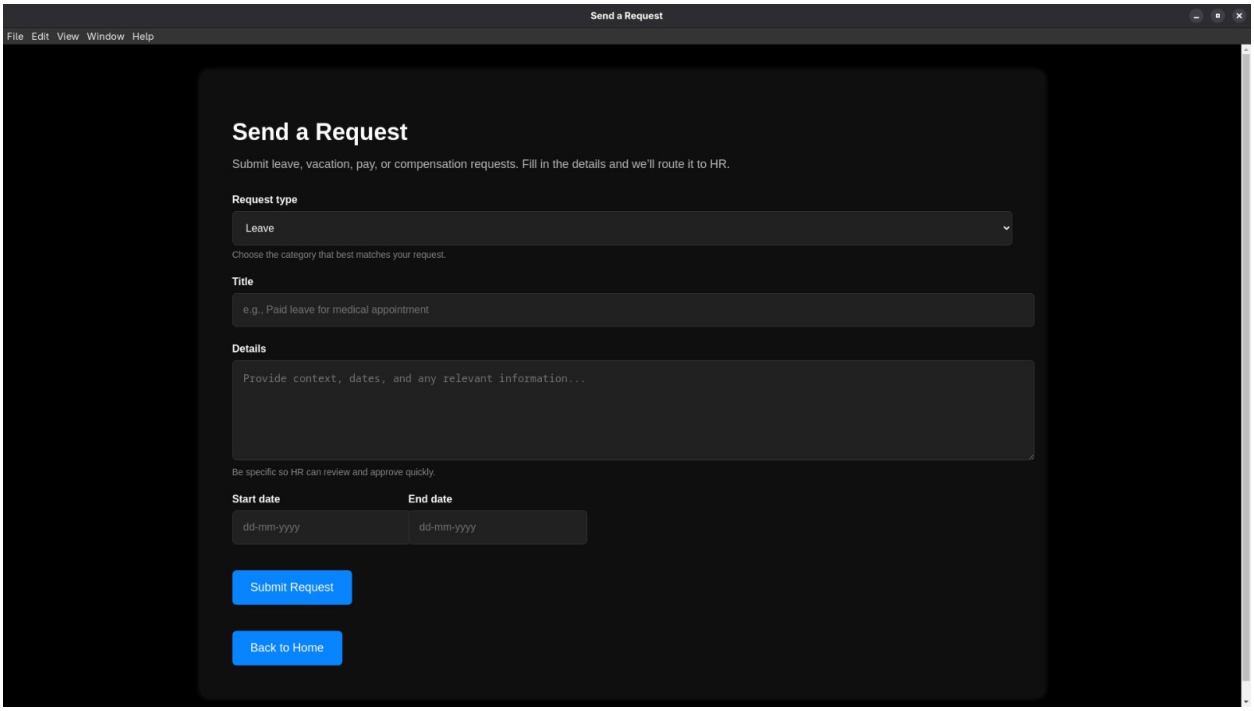


Figure 4.10 Send Request Page

4.5 Security Issues

1. Unauthorized Access

- **Issue**: Access to restricted data
- **Solution**: JWT authentication

2. Password Security

- **Issue**: Weak password storage
- **Solution**: bcrypt hashing

3. Data Exposure

- **Issue**: Sensitive data leaks
- **Solution**: Use environment variables (.env)

4. API Security

- **Issue**: Unprotected routes
- **Solution**: Middleware protection

4.6 Test Cases

Login Test Cases

Test Case ID	Scenario	Expected Result
TC_LOGIN_01	Valid Login	Dashboard opens
TC_LOGIN_02	Invalid Login	Error message

Table 4.6: Login Test Cases

Task Module Test Cases

Test Case ID	Scenario	Expected Result
TC_TASK_01	Add Task	Task saved
TC_TASK_02	Delete Task	Task removed

Table 4.7: Task Module Test Cases

Project Module Test Cases

Test Case ID	Scenario	Expected Result
TC_PROJ_01	Create Project	Project added
TC_PROJ_02	Update Status	Status updated

Table 4.8: Project Module Test Cases

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Details

The HERMES – Employee Management System is implemented using a full-stack architecture consisting of frontend, backend, database, and desktop application layers.

5.1.1 Frontend Implementation

The frontend is developed using HTML, CSS, and JavaScript, providing an interactive and user-friendly interface.

Key functionalities:

- User Login and Registration
- Dashboard view
- Task, Team, and Project management
- Request submission
- The frontend communicates with the backend using REST APIs.

5.1.2 Backend Implementation

The backend is developed using Node.js and Express.js, which handles all business logic and API requests.

Main responsibilities:

- Handling user authentication
- Managing CRUD operations
- Validating user requests
- Connecting with MongoDB database

5.1.3 Database Implementation

The system uses MongoDB as the database, with Mongoose for schema modeling.

Main collections:

- Users
- Tasks
- Teams
- Projects
- Requests

5.1.4 Desktop Application

The system is integrated with Electron.js to provide a desktop-based application interface.

Features:

- Native desktop experience
- Smooth page navigation
- Easy access to all modules

5.2 Sample Code Snippets

Login API (Authentication):

```
app.post("/login", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.json({ message: "User not found" });
  const valid = await bcrypt.compare(req.body.password, user.password);
  if (!valid) return res.json({ message: "Invalid password" });
  const token = jwt.sign({ id: user._id }, process.env.SECRET_KEY);
  res.json({ message: "Login successful", token });
});
```

Task Creation API:

```
app.post("/add_task", async (req, res) => {
  const task = new Task(req.body);
  await task.save();
  res.json({ message: "Task added successfully" });
});
```

Mongoose Schema Example:

```
const taskSchema = new mongoose.Schema({
  title: String,
  description: String,
  duration: Number,
  date: Date
});
```

5.3 Testing

5.3.1 Types of Testing

1. Unit Testing

Individual modules are tested separately.

Example: Task creation module

2. Integration Testing

Testing interaction between modules.

Example: Login → Dashboard → Task operations

3. System Testing

Entire system is tested as a whole.

4. User Acceptance Testing (UAT)

System is tested by users to ensure it meets expectations.

5.3.2 Test Cases

Test Case ID	Module	Input	Expected Output	Result
TC-01	Login	Valid data	Login successful	Pass
TC-02	Login	Invalid data	Error message	Pass
TC-03	Task	Add task	Task created	Pass
TC-04	Project	Update	Updated successfully	Pass
TC-05	Request	Submit	Request stored	Pass

Table 5.1: Test Cases

5.4 Tools Used for Testing

- Postman / Thunder Client – API testing
- MongoDB Compass – Database verification
- Browser Developer Tools – Debugging

5.5 Error Handling

The system includes proper error handling mechanisms:

- Invalid credentials → Error message
- Unauthorized access → Token validation
- Server errors → Try-catch handling
- Database errors → Proper response

5.6 Security Implementation

Security features implemented in the system include:

- Password hashing using bcrypt
- JWT-based authentication
- Protected API routes
- Environment variables for sensitive data

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Results

The HERMES – Employee Management System was successfully developed and tested. The system provides a complete solution for managing employees, tasks, teams, and projects in an organization.

The application performs all the required functionalities efficiently, including authentication, task management, team handling, project tracking, and request processing.

Key Outputs of the System

- Users can register and log in securely
- Tasks can be created, viewed, and deleted
- Teams can be managed and members assigned
- Projects can be tracked with status updates
- Requests can be submitted and processed
- Dashboard shows task summary and analytics

6.2 System Interface Description

The HERMES system provides a well-structured and user-friendly interface that allows users to interact efficiently with different modules of the application.

The system interface is divided into the following components:

1. User Authentication Interface

- Enables users to register and log in securely
- Uses JWT-based authentication for access control

2. Dashboard Interface

- Acts as the main control panel of the system

- Provides quick access to all modules

3. Task Management Interface

- Allows users to create, view, and delete tasks
- Displays task-related information clearly

4. Team Management Interface

- Provides functionality to create and manage teams
- Displays team members and structure

5. Project Management Interface

- Allows creation and tracking of projects
- Supports status updates (To Do, In Progress, Completed)

6. Request Management Interface

- Enables submission of requests such as leave or issues
- Maintains proper workflow

7. Analytics Interface

- Displays task summary and work analysis
- Helps in monitoring performance

6.3 Discussion

The developed system improves the efficiency of employee management by replacing manual processes with an automated solution.

- Advantages of the System
- Centralized data management
- Improved task tracking
- Better team collaboration
- Secure authentication system
- User-friendly interface

Limitations

- System is deployed locally (no cloud deployment)
- Limited role-based access control
- No real-time notifications

6.4 Future Enhancements

The system can be further improved by adding:

- Role-based access control (Admin/User)
- Cloud deployment
- Real-time updates using WebSockets
- Notification system
- File upload feature

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The **HERMES – Employee Management System** was successfully designed and implemented to improve the efficiency of managing employees, tasks, teams, and projects within an organization.

The system provides a centralized and secure platform where users can perform various operations such as authentication, task management, project tracking, and request handling. By automating these processes, the system reduces manual effort, minimizes errors, and enhances overall productivity.

The application is developed using modern technologies such as **HTML, CSS, JavaScript, Node.js, Express.js, MongoDB, and Electron**, ensuring a reliable and scalable solution. Proper testing has been conducted to ensure that all modules function correctly.

Overall, the system achieves its objective of providing an efficient, user-friendly, and secure employee management solution.

7.2 Future Scope

Although the system performs effectively, there are several enhancements that can be implemented in the future:

- Implementation of **role-based access control** (Admin, Manager, Employee)
- Deployment on **cloud platforms** for better accessibility
- Integration of **real-time features** using WebSockets
- Addition of **notification and alert system**
- Development of a **mobile application version**
- Implementation of **file upload and document management system**
- Enhancement of **UI/UX design** for better user experience

REFERENCES

The following resources were used during the development of the HERMES – Employee Management System:

1. Node.js Official Documentation – https://nodejs.org
2. Express.js Documentation – https://expressjs.com
3. MongoDB Documentation – https://www.mongodb.com/docs
4. Mongoose Documentation – https://mongoosejs.com
5. Electron.js Documentation – https://www.electronjs.org/docs
6. JSON Web Token (JWT) Documentation – https://jwt.io
7. bcrypt Documentation – https://www.npmjs.com/package/bcrypt
8. MDN Web Docs (HTML, CSS, JavaScript) – https://developer.mozilla.org
9. Stack Overflow – https://stackoverflow.com
10. Various online tutorials and developer resources

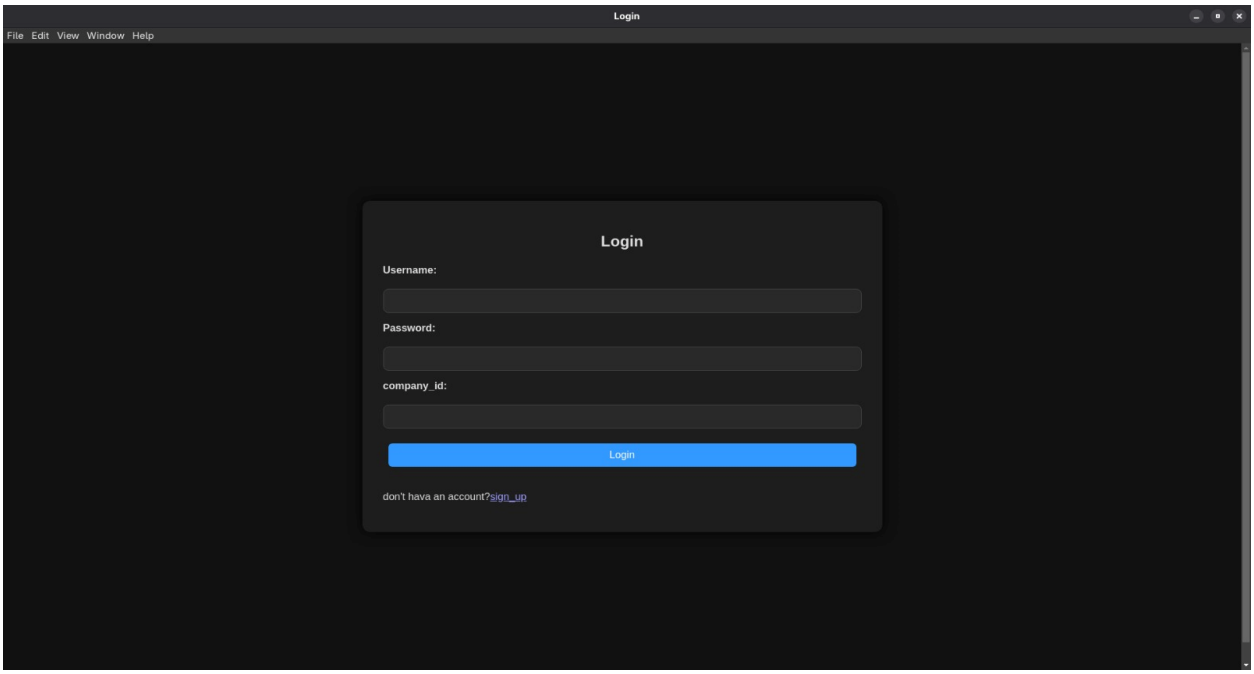
7.3 GLOSSARY

Term	Description
EMS	Employee Management System
JWT	JSON Web Token used for secure authentication
API	Application Programming Interface
CRUD	Create, Read, Update, Delete operations
MongoDB	NoSQL database used in the project
Express.js	Backend framework for Node.js
Node.js	JavaScript runtime environment
Electron	Framework to build desktop applications
bcrypt	Library for password hashing
Frontend	User interface part of the application
Backend	Server-side logic of the application
Database	Storage system for application data

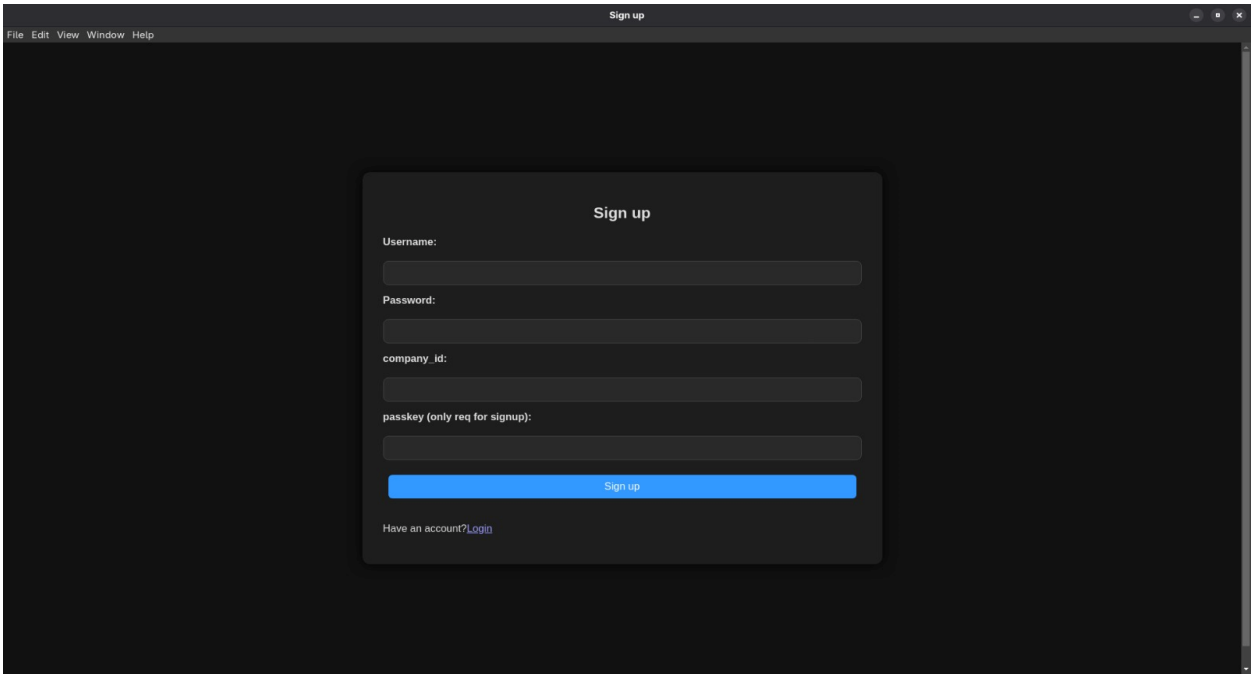
APPENDIX

Appendix A: Project Screenshots

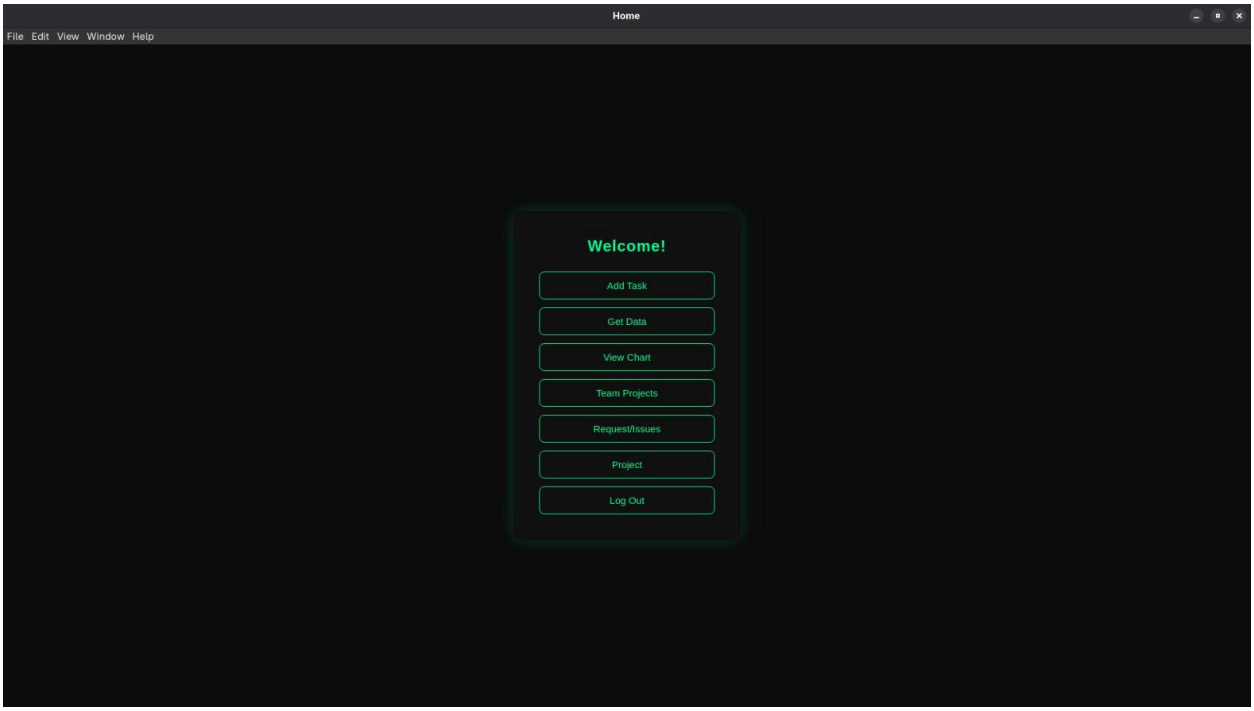
- **Login page**



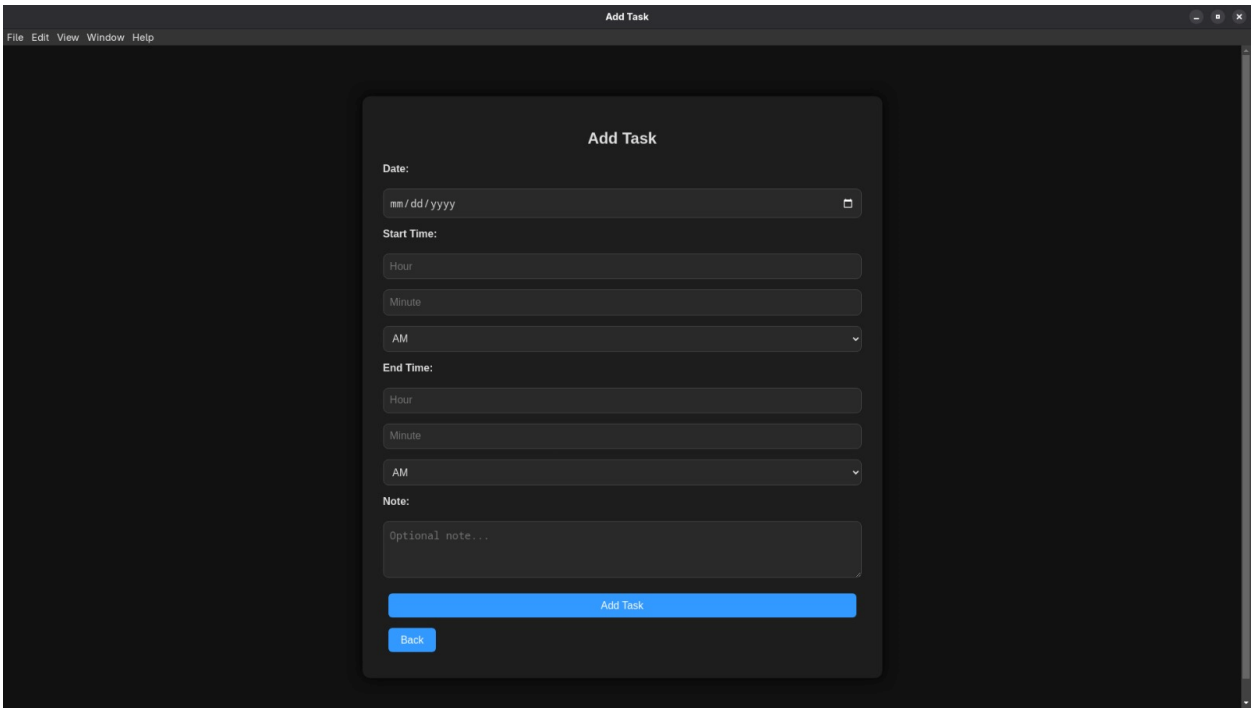
- **Sign up page**



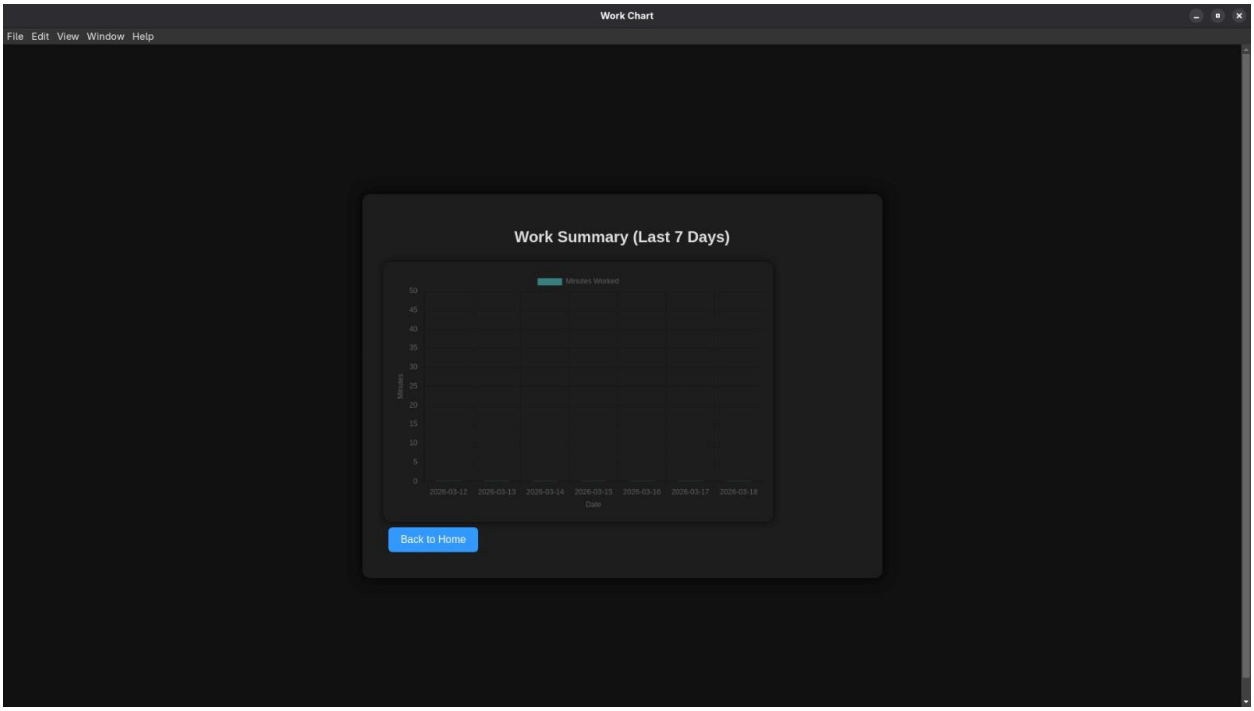
- **Home page**



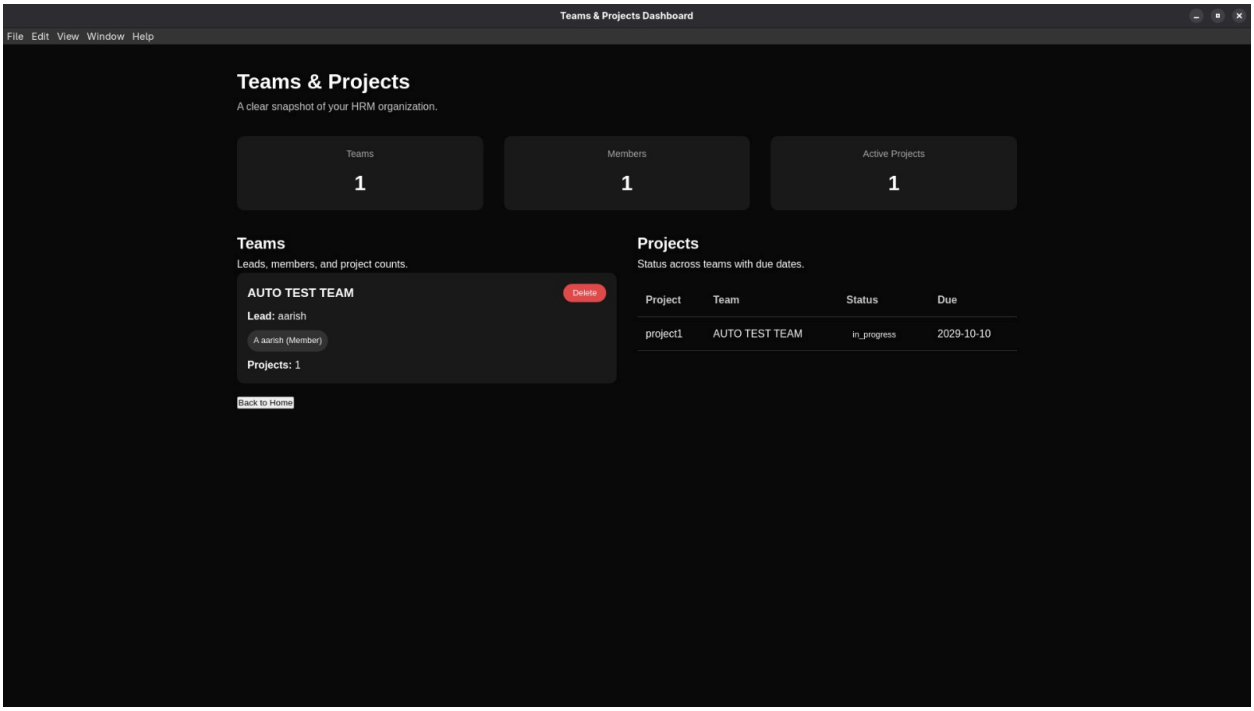
- **Add Task Page**



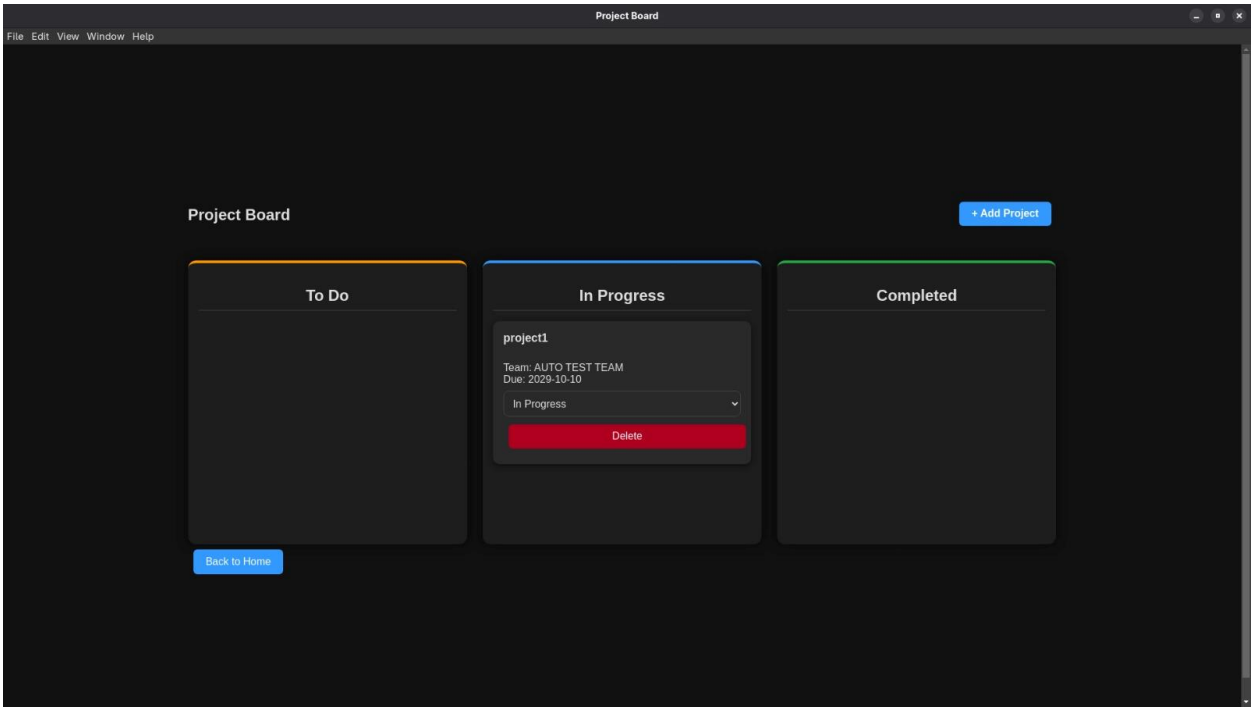
- **Work Summary Page**



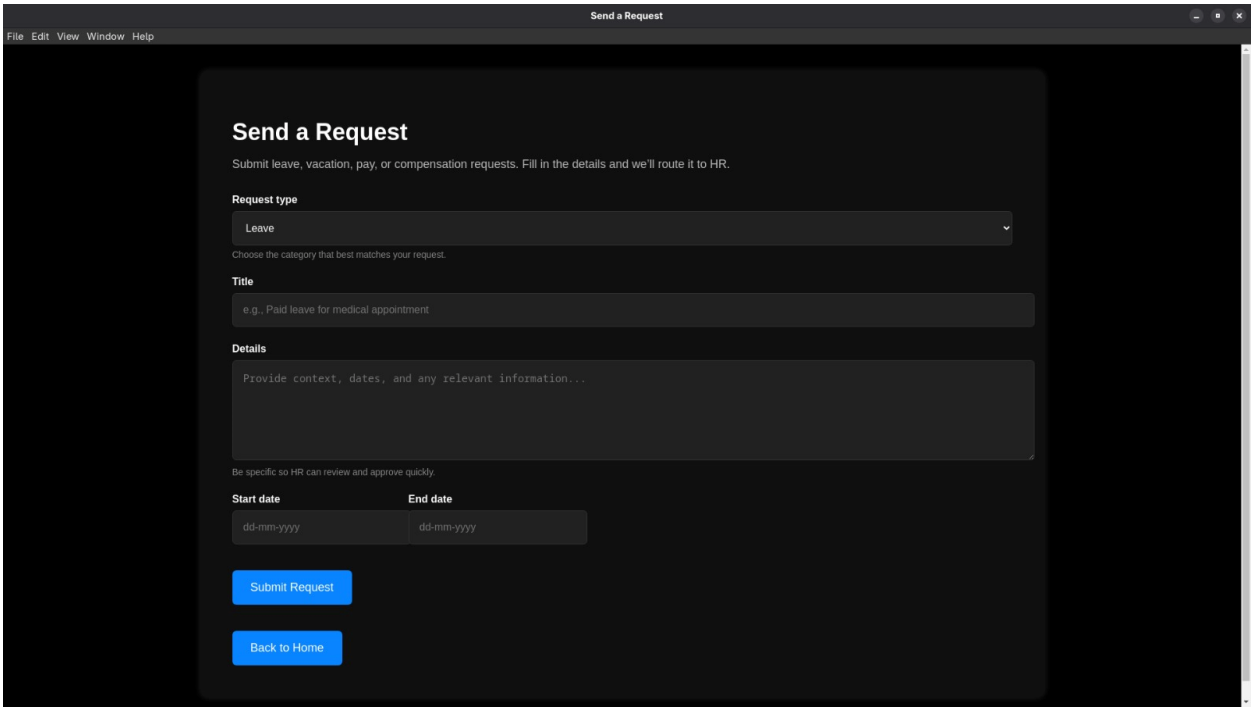
- **Teams & Projects Page**



- **Project Borad Page**



- **Send Request Page**



Appendix B: Sample Environment File

- MONGO_URL=your_mongodb_connection_string
- SECRET_KEY=your_secret_key

Appendix C: System Requirements

Hardware Requirements:

- Minimum 8 GB RAM
- Processor: Intel i3 or above
- Internet connection

Software Requirements:

- Operating System: Windows/Linux
- Node.js installed
- MongoDB installed
- VS Code